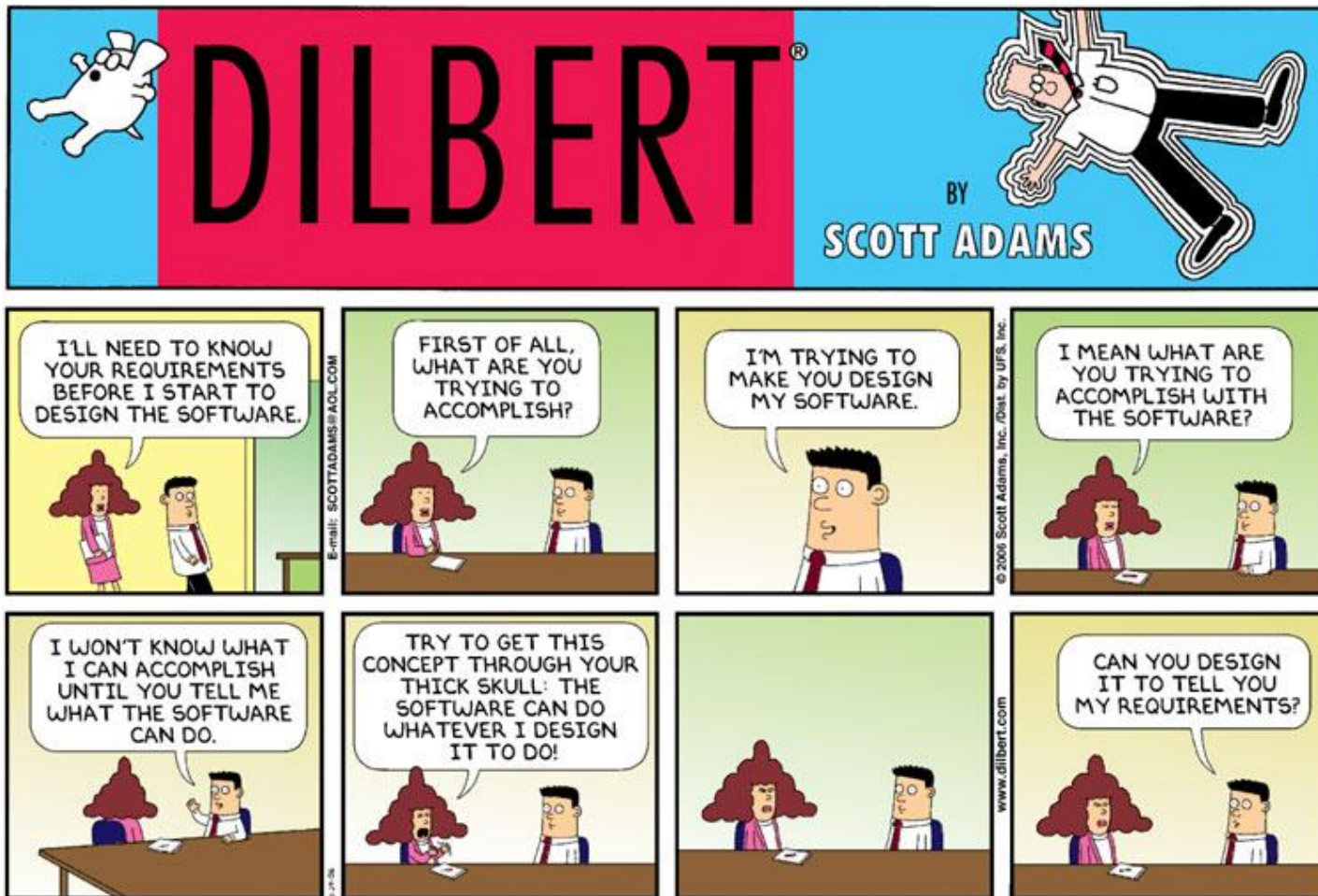
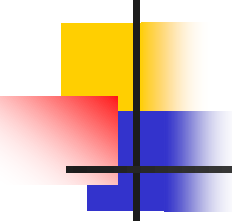


Use-Case Diagrams





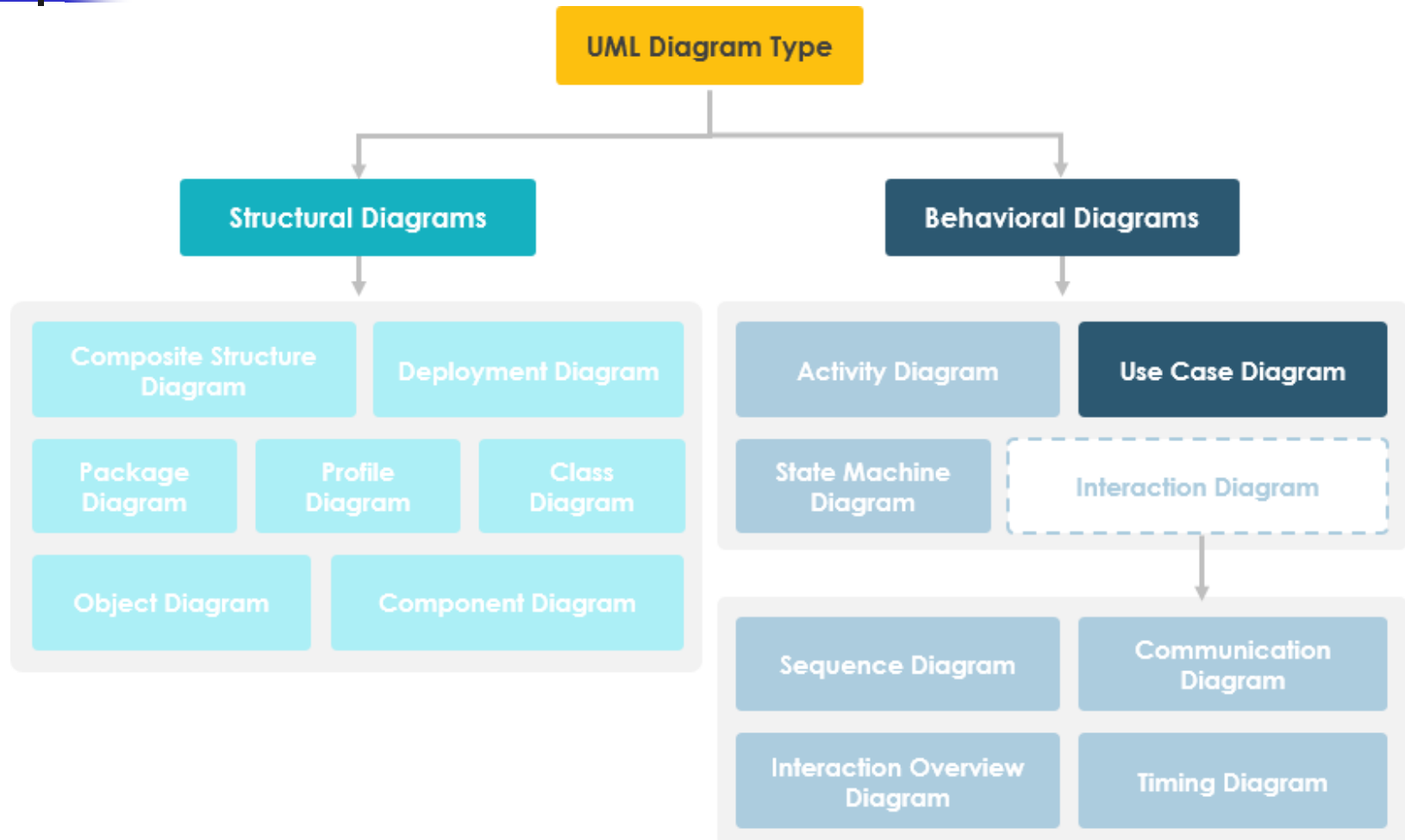
How to use UML diagrams to design software system?

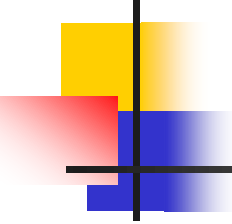
■ Types of UML Diagrams:

- Use Case Diagram
- System Sequence diagram
- Domain Model
- Class Diagram
- Sequence Diagram
- State Diagram

This is only a subset of diagrams ... but are most widely used

UML Diagrams



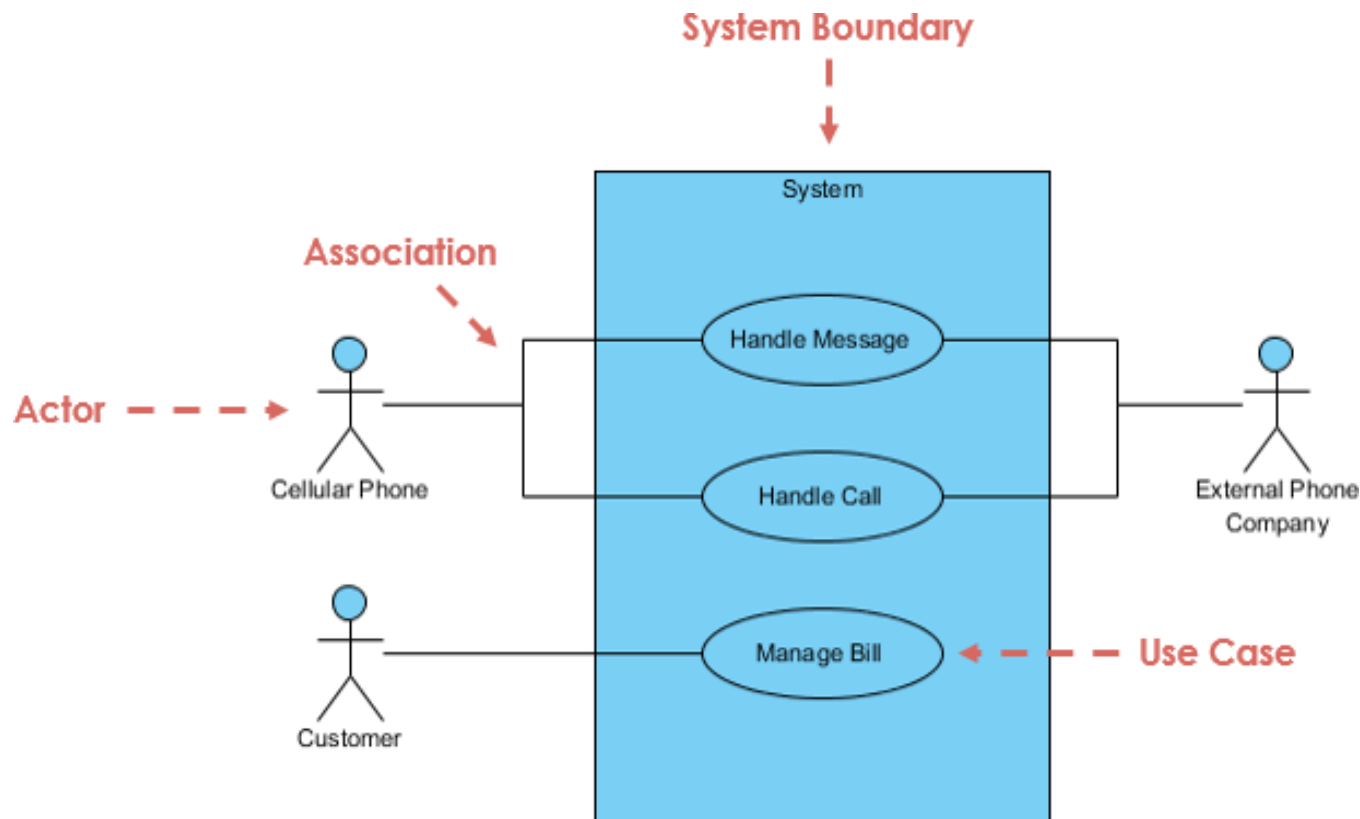


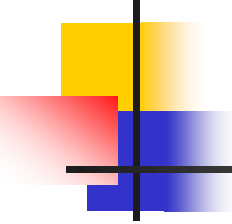
Use-Case Diagrams

- A use-case diagram is a set of use cases
- A use case is a model of the interaction between
 - External users of a software product (actors) and
 - More precisely, an actor is a user playing a specific role
- describing a set of user **scenarios**
- capturing user requirements
- **contract** between end user and software developers

Use case diagram

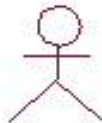
- A standard form of use case diagram is defined in the Unified Modeling Language as shown in the Use Case Diagram example below:



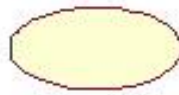


Use-Case Diagrams

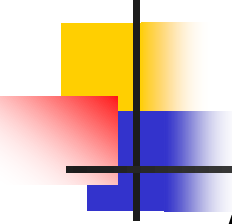
- **Actors:** A role that a user plays with respect to the system, including human users and other systems. e.g., inanimate physical objects (e.g. robot); an external system that needs some information from the current system.
- **Use case:** A set of scenarios that describing an interaction between a user and a system, including alternatives.
- **System boundary:** rectangle diagram representing the boundary between the actors and the system.



Actor



Use Case



Notation description

Actors are **roles** played by people, organisations, other software and devices that interact with the application.

Primary actors

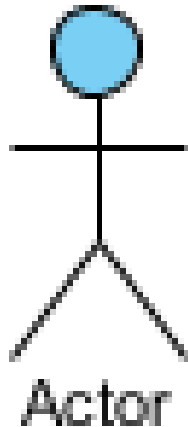
- Those that have user goals fulfilled through using services of the system

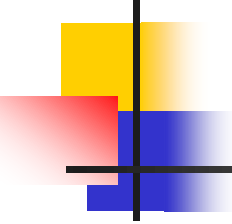
Supporting actors

- Provides a service to the system

Offstage actor

- has an interest in the behaviour of the use case, but is not primary or supporting. (e.g. tax agency)



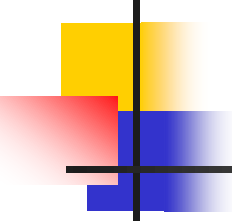


Use case

- System function (process - automated or manual)
- Named by verb + Noun (or Noun Phrase).
- i.e. Do something
- Each Actor must be linked to a use case, while some use cases may not be linked to actors.



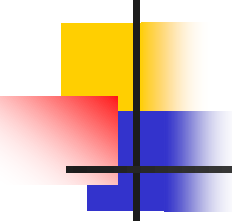
Use Case



Communication link

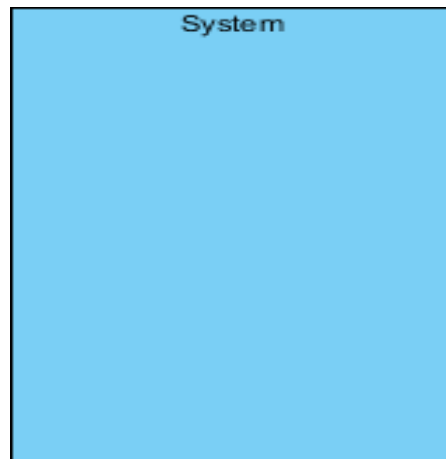
- The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link.
- Actors may be connected to use cases by associations, indicating that the actor and the use case communicate with one another using messages.





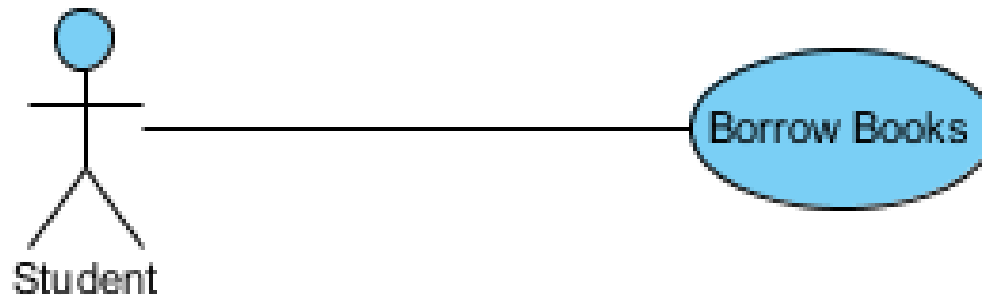
Boundary of the system

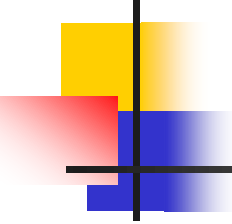
- The system boundary is potentially the entire system as defined in the requirements document.
- For large and complex systems, each module may be the system boundary.
- For example, for an ERP system for an organization, each of the modules such as personnel, payroll, accounting, etc. can form a system boundary for use cases specific to each of these business functions.
- The entire system can span all of these modules depicting the overall system boundary



Use case - example

- **Association Link**
- A Use Case diagram illustrates a set of use cases for a system, i.e. the actors and the relationships between the actors and use cases.



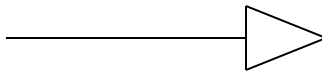


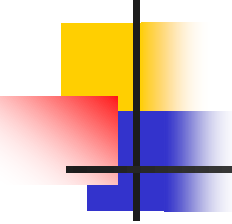
Use-Case Diagrams

- Association: communication between an actor and a use case; Represented by a solid line.



- Generalization: relationship between one general use case and a special use case (used for defining special alternatives) Represented by a line with a triangular arrow head toward the parent use case.

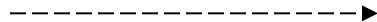




Use-Case Diagrams

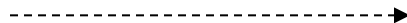
Include: a dotted line labeled <<include>> beginning at base use case and ending with an arrow pointing to the include use case. The include relationship occurs when a chunk of behavior is similar across more than one use case. Use “include” in stead of copying the description of that behavior.

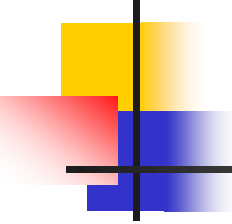
<<include>>



Extend: a dotted line labeled <<extend>> with an arrow toward the base case. The extending use case may add behavior to the base use case. The base class declares “extension points”.

<<extend>>





Use case relationship

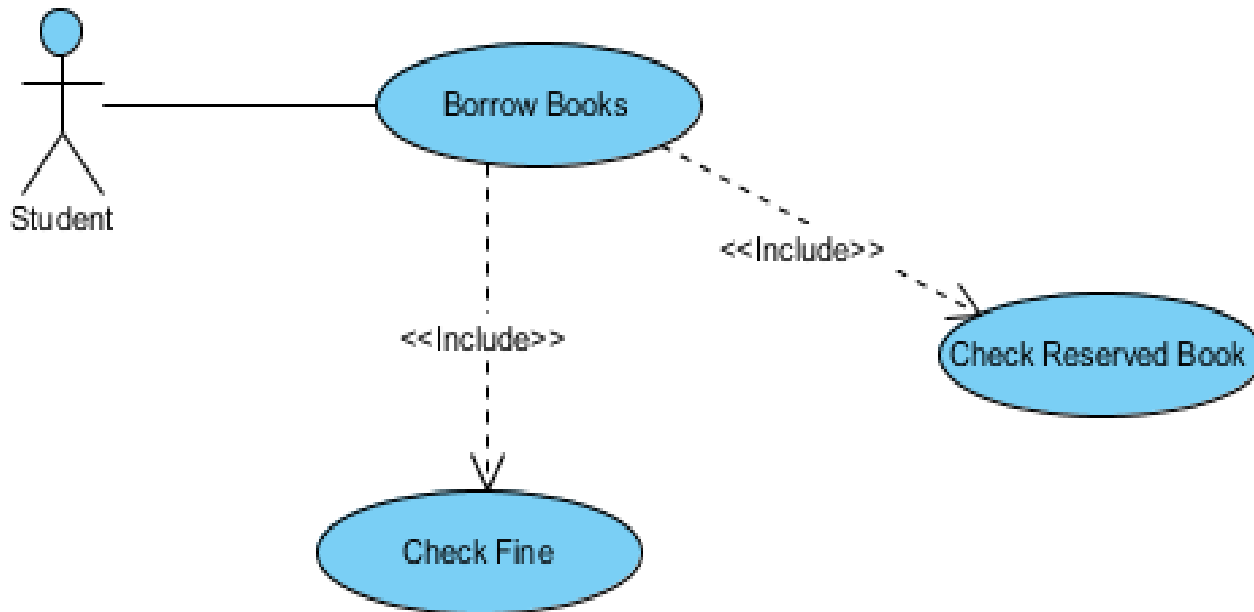
- **Include**
- When a use case is depicted as using the functionality of another use case, the relationship between the use cases is named as include or uses relationship.
- A use case includes the functionality described in another use case as a part of its business process flow.
- A uses relationship from base use case to child use case indicates that an instance of the base use case will include the behavior as specified in the child use case.
- An include relationship is depicted with a directed arrow having a dotted line. The tip of arrowhead points to the child use case and the parent use case connected at the base of the arrow.
- The stereotype "<<include>>" identifies the relationship as an include relationship.



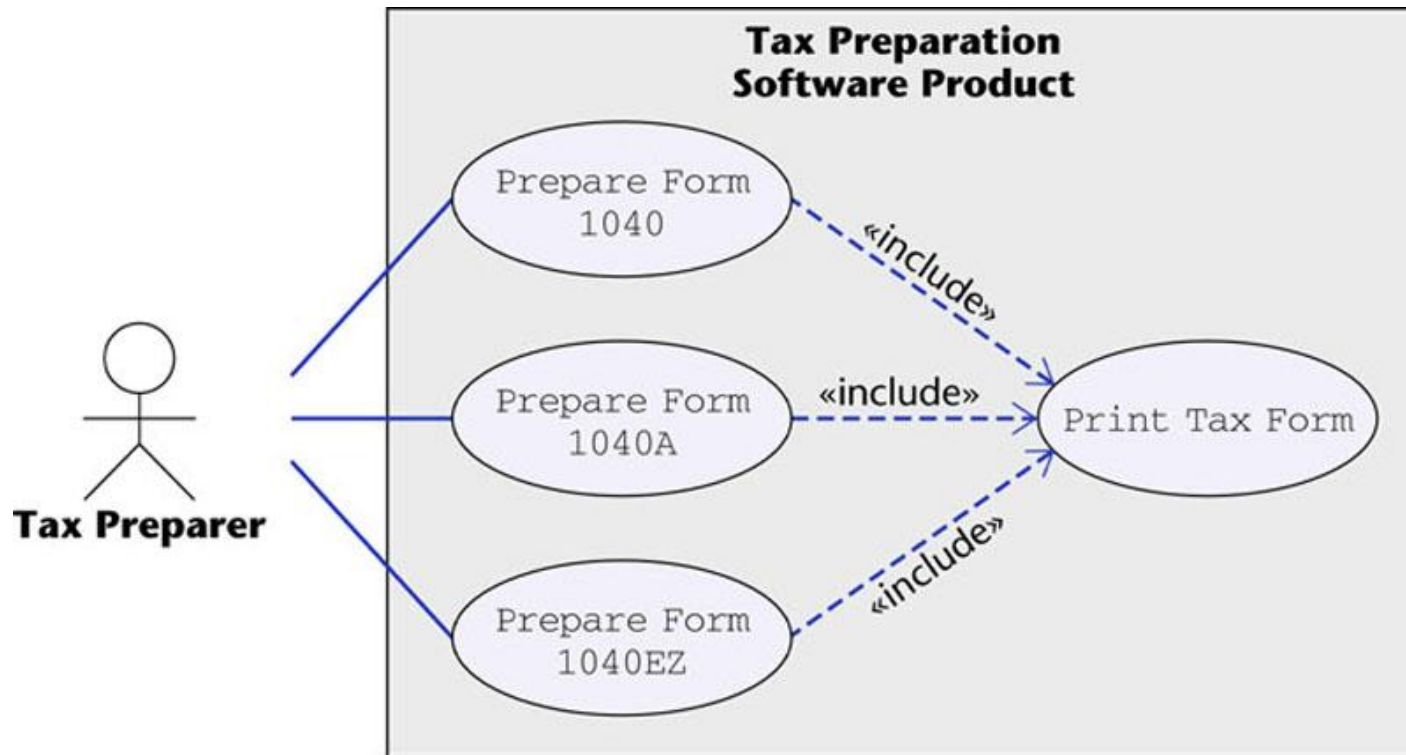
Use case - example

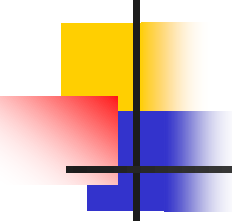
- **Include link**

- The include relationship adds additional functionality not specified in the base use case. The <<Include>> relationship is used to include common behavior from an included use case into a base use case in order to support the reuse of common behavior.



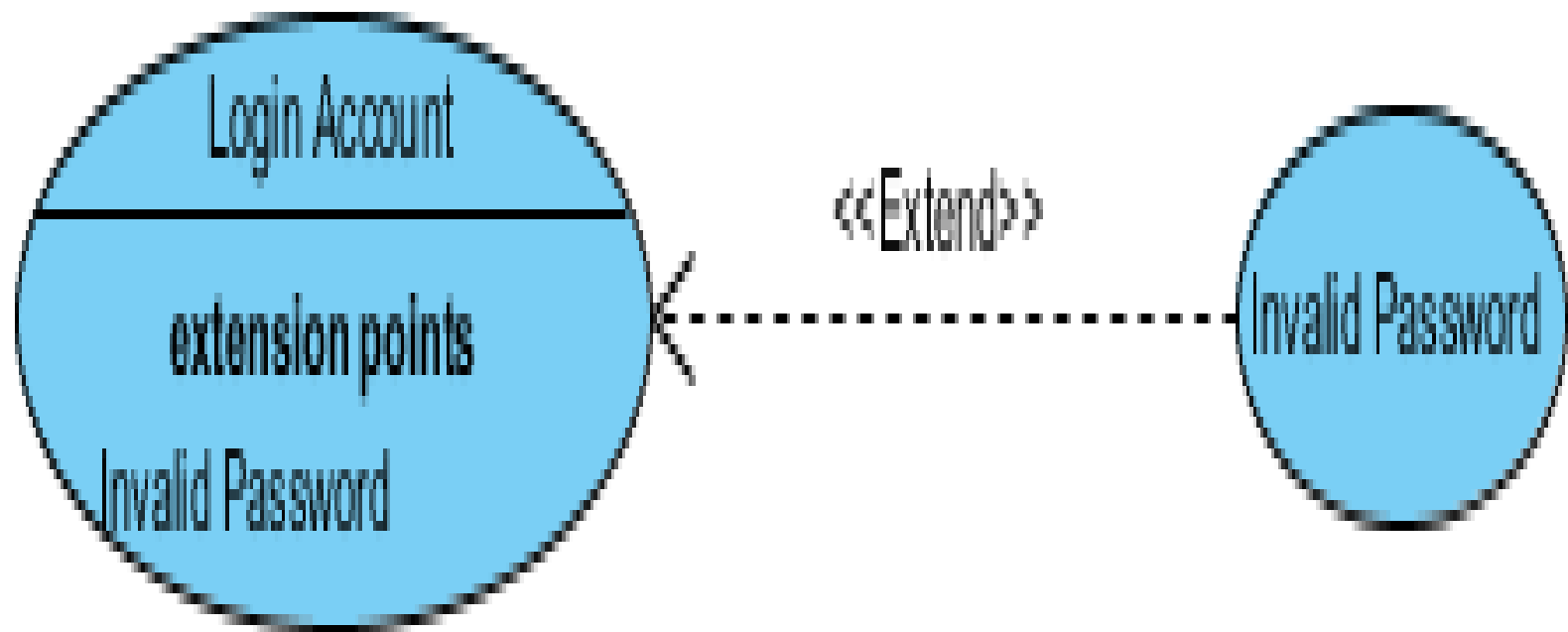
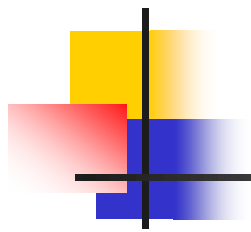
Use-Case Diagrams





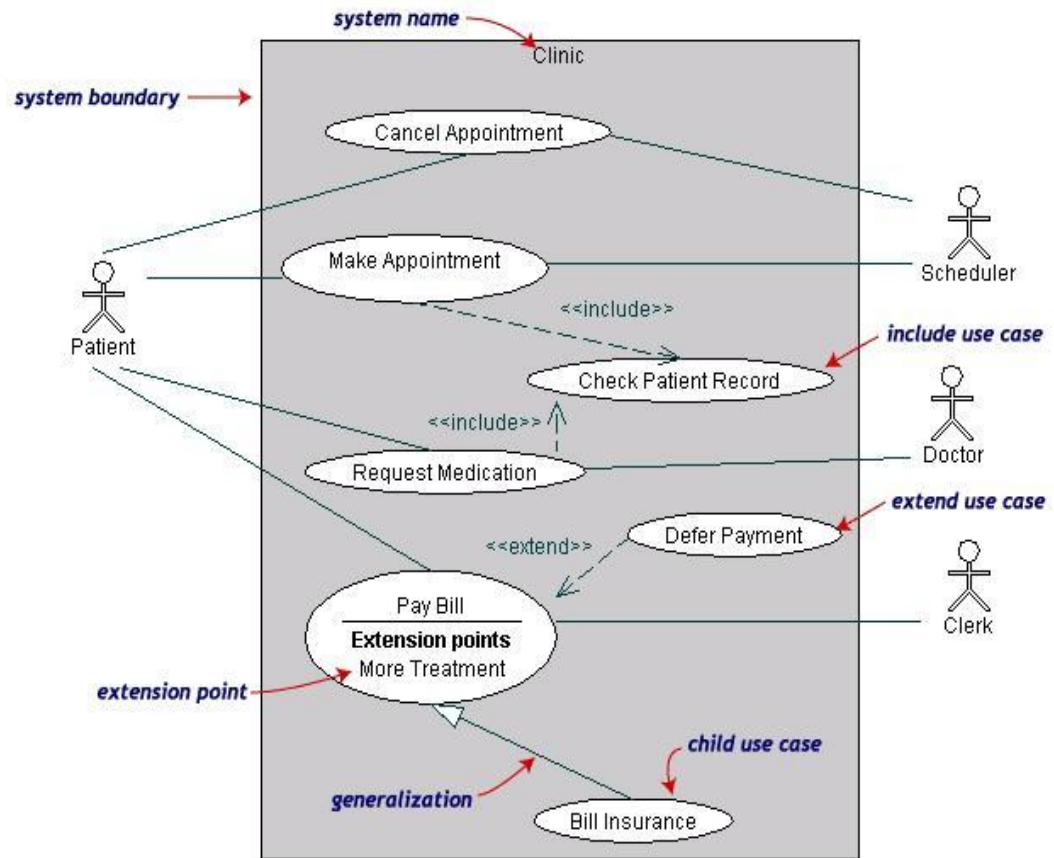
Use case relationship

- **Extends**
 - indicates that an "Invalid Password" use case may include (subject to specified in the extension) the behavior specified by base use case "Login Account".
 - Depict with a directed arrow having a dotted line. The tip of arrowhead points to the base use case and the child use case is connected at the base of the arrow.
 - The stereotype "<<extends>>" identifies as an extend relationship

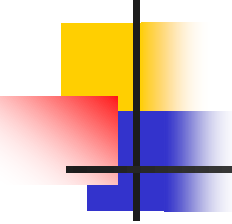


Use-Case Diagrams

- Both **Make Appointment** and **Request Medication** include **Check Patient Record** as a subtask (include)
- The **extension point** is written inside the base case **Pay bill**; the extending class **Defer payment** adds the behavior of this extension point. (extend)
- **Pay Bill** is a parent use case and **Bill Insurance** is the child use case. (generalization)



(TogetherSoft, Inc)



Use case tips

Now, check the tips below to see how to apply use case effectively in your software project.

- Always structure and organize the use case diagram from the perspective of actors.
- Use cases should start off simple and at the highest view possible. Only then can they be refined and detailed further.
- Use case diagrams are based upon functionality and thus should focus on the "what" and not the "how".

Note that:

- While a use case itself might drill into a lot of detail about every possibility, a use-case diagram is often used for a higher-level view of the system as blueprints.
- A use case is complete without it's extent
- A use case is not complete withput it's include